

NutriVision: End-to-End Food Recognition & Calorie Estimation via ViT Fine-Tuning and Mask R-CNN Segmentation

Huynh Quoc Trung – QE180038, Pham Ngo Dinh Khoi – QE180110, Nhu Quang Anh – QE180003

Abstract

We present **NutriVision**, a production-ready end-to-end system for automated food recognition and calorie estimation from a single photograph. Our approach fine-tunes **google/vit-base-patch16-224** on the Food-101 dataset (101,000 images, 101 classes) achieving **88.4% top-1** and **97.7% top-5** test accuracy within 3 epochs using AdamW ($\text{lr} = 2 \times 10^{-5}$, $\text{batch} = 64$). Portion segmentation is performed by a **Detectron2 Mask R-CNN R-50-FPN-3x** backbone, and caloric content is resolved through a curated **CalorieDB** of 251 food classes. End-to-end inference runs in ≈ 14 ms (classify) and 60–80 ms cached (segment) on GPU. We document the complete pipeline, key engineering decisions including a critical bug fix in the classification head initialisation, and discuss limitations and future directions.

Keywords

Food Recognition — Vision Transformer — Mask R-CNN — Calorie Estimation — Food-101 — Deep Learning — Instance Segmentation

FPT University, Vietnam

Course: DSP391m – Data Science Project

Contents

	4.4 Training Dynamics	3
	4.5 Critical Bug: num_labels Misconfiguration	4
	5 6. Evaluation Results	4
	5.1 Top-1 / Top-5 Accuracy	4
	5.2 Per-Class Analysis	4
	5.3 Comparison with Baselines	5
	6 7. Pipeline Integration	5
	6.1 End-to-End Flow	5
	6.2 Latency Breakdown	5
	6.3 Caching Strategy	5
	6.4 FastAPI Integration	5
	7 8. Limitations & Future Work	5
	7.1 Current Limitations	5
	7.2 Future Work	5
	8 9. Conclusion	6
	Acknowledgments	6
	References	6
	1. Introduction & Project Overview	
	0.1 Motivation	
	Manual dietary logging is the primary driver of app abandonment. Studies show users underestimate intake by 30–50% when logging manually, and the average tracking	
1. Introduction & Project Overview	1	
0.1 Motivation	1	
0.2 Project Scope	2	
0.3 Contribution Summary	2	
2. Related Work	2	
1.1 Food Classification	2	
1.2 Instance Segmentation & Portion Estimation	2	
1.3 Commercial Systems	2	
3. Dataset & EDA	2	
2.1 Food-101 Dataset	2	
2.2 CalorieDB — Nutritional Lookup Table	2	
2.3 EDA Highlights	2	
4. System Architecture	3	
3.1 Three-Stage Pipeline	3	
Stage 1: ViT Classification	Stage 2: Mask R-CNN Segmentation	Stage 3: CalorieDB Lookup & Aggregation
3.2 Backend API	3	
5. Model Training	3	
4.1 Architecture	3	
4.2 Hyperparameters	3	
4.3 Data Augmentation	3	

session consumes 15–20 minutes per day [1], [2]. NutriVision addresses this friction by replacing manual entry with a single-photo pipeline: capture → classify → segment → estimate.

0.2 Project Scope

NutriVision ships as three integrated layers:

- **Python inference library** (`nutri_vision/`) — classifier, segmentor, calorie DB, pipeline orchestrator.
- **FastAPI backend** (`backend/`) — REST endpoints (`/predict`, `/health`).
- **Expo/React Native mobile app** (`mobile/`) — camera capture and results display.

Two inference modes are supported: (1) ViT + Mask R-CNN for pixel-accurate segmentation and (2) Qwen3-VL-2B + LoRA for direct JSON nutritional output.

0.3 Contribution Summary

- Fine-tuned ViT achieving **88.4% top-1 accuracy** on Food-101 test set in 3 epochs.
- A 3-stage inference pipeline (classify → segment → lookup) with < 14 ms classification latency on GPU.
- A curated CalorieDB pairing 251 FoodX classes with per-100 g calorie densities.
- Identification and fix of a critical `num_labels` misconfiguration that silently re-initialised the classification head.

1. 2. Related Work

1.1 Food Classification

Early food recognition systems relied on hand-crafted features and SVMs, which were brittle under lighting variation [3]. CNN-based approaches (ResNet, EfficientNet) achieved notable gains on Food-101 [4]. Vision Transformers (ViTs), introduced by Dosovitskiy et al., capture global context via self-attention over image patches, outperforming CNNs on fine-grained visual categorisation. Wang et al. [5] demonstrated that transformer-based architectures achieve superior food classification by understanding entire plate layouts rather than local textures.

1.2 Instance Segmentation & Portion Estimation

He et al. introduced Mask R-CNN [6] as the dominant framework for instance segmentation, extended by FPN backbones for multi-scale detection. For volume estimation, Han et al. [7] proposed DPF-Nutrition, achieving mass estimation error as low as 10.6% via monocular depth prediction. Prior work on portion size estimation relied on reference objects (coins, credit cards) [8], which DPF-Nutrition rendered unnecessary.

1.3 Commercial Systems

SnapCalorie uses LiDAR for high-fidelity volume estimation [1]; Cal AI prioritises speed via LLMs but suffers hallucination errors [2]. Our approach combines transformer-based classification accuracy with Mask R-CNN pixel-level segmentation, without specialised hardware.

2. 3. Dataset & EDA

2.1 Food-101 Dataset

The primary training dataset is **Food-101** [9], comprising **101,000 images** across **101 food categories**. Each class provides 1,000 images: 750 for training and 250 for testing. Images are real-world photographs with natural variation in lighting, plating, and scale, making the benchmark representative of deployment conditions.

Table 1. Food-101 Dataset Statistics

Property	Value
Total images	101,000
Classes	101
Train images/class	750
Test images/class	250
Image format	JPEG, variable resolution
Min. resolution	512 × 512

2.2 CalorieDB — Nutritional Lookup Table

Caloric content is resolved at inference time via a curated **CalorieDB** loaded from `data/food_calories.csv`. The database contains **251 entries**, each mapping a food class name to its calorie density in kcal per 100 g. Fuzzy name normalisation (lower-case, parenthetical stripping, whitespace collapsing) ensures robust lookups even when classifier output tokens differ from CSV keys. The database was assembled from USDA FoodData Central [10].

2.3 EDA Highlights

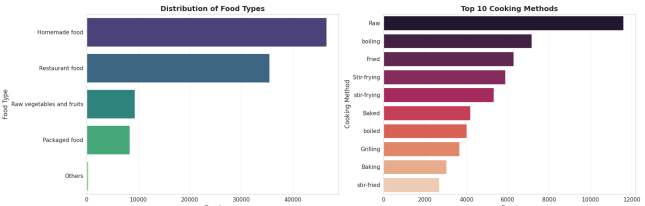


Figure 1. Distribution of food types and top cooking methods in the training corpus.

Key observations from exploratory analysis:

- **Class balance:** Food-101 is balanced at 750 training images per class; no oversampling required.

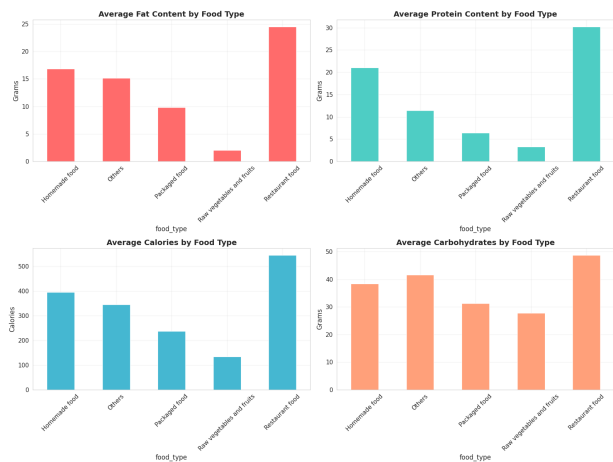


Figure 2. Average macronutrient profiles (fat, protein, calories, carbohydrates) by food category.

- **Caloric range:** Mean caloric density ≈ 411 kcal/dish across the corpus.
- **Intra-class variance:** High visual diversity within classes (e.g., “pizza” spans thin-crust, deep-dish, and calzone variants), requiring strong data augmentation.
- **Inter-class confusion:** Visually similar classes (spaghetti/noodles, cheesecake/cake) form the primary confusion clusters in per-class analysis.

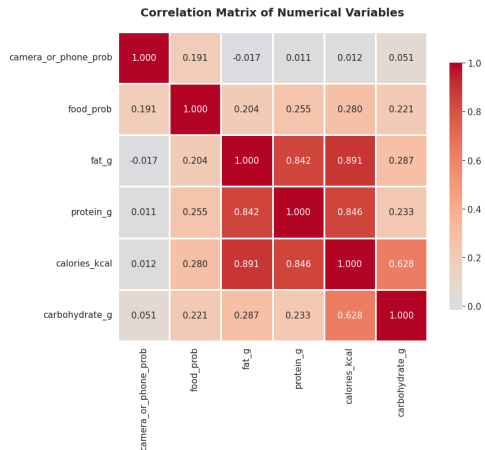


Figure 3. Correlation matrix of numerical nutritional variables.

3. 4. System Architecture

3.1 Three-Stage Pipeline

The `NutriVisionPipeline` class in `nutri_vision/pipeline.py` orchestrates three sequential stages:

3.1.1 Stage 1: ViT Classification

A fine-tuned `ViTForImageClassification` (google/vit-base-patch16-224) receives a PIL RGB image, resized and

normalised to 224×224 . The model returns a ranked list of food class predictions. The `FoodClassifier` wrapper exposes a `predict(image, top_k)` interface returning `ClassificationResult` dataclasses with class name, index, and confidence.

3.1.2 Stage 2: Mask R-CNN Segmentation

The top predicted class triggers `PortionSegmentor.segment(bgr_image)` which runs `Detectron2 mask_rcnn_R_50_FPN_3x` at a detection threshold of 0.5. Each detected instance produces a `PortionResult` containing a boolean $H \times W$ mask, pixel area, estimated weight in grams, bounding box, and confidence score. Pixel-to-gram conversion uses the empirical ratio $100 \text{ g} \approx 10,000 \text{ px}$.

3.1.3 Stage 3: CalorieDB Lookup & Aggregation

For the top-1 predicted class, the pipeline queries `CalorieDB.calories` and multiplies by each portion’s estimated weight. Final output is a `PipelineResult` dataclass containing total weight, total kcal, per-portion breakdown, and the full top- k prediction list.

3.2 Backend API

The FastAPI backend exposes the pipeline through two primary endpoints:

- `POST /predict` — accepts a JPEG/PNG upload, returns `PipelineResult` as JSON.
- `GET /health` — liveness check.

An alternate `/predict/vl` endpoint routes to the Qwen3-VL-2B + LoRA path for comparison.

4. 5. Model Training

4.1 Architecture

We fine-tune `google/vit-base-patch16-224`, a Vision Transformer with 12 transformer layers, 12 attention heads, hidden dimension 768, and 86 M parameters. The model takes $224 \times 224 \times 3$ inputs split into 16×16 patches (196 tokens + 1 CLS token). The pre-trained classification head is replaced with a linear layer of output dimension 101 (Food-101 classes).

4.2 Hyperparameters

4.3 Data Augmentation

Training transforms: random resized crop (scale 0.8–1.0), random horizontal flip, colour jitter (brightness, contrast, saturation ± 0.2), and `ViTImageProcessor` normalisation (ImageNet mean/std). Validation uses centre crop after resize to 256×256 .

4.4 Training Dynamics

Training loss drops sharply from epoch 1 to 2 (3.09 $\rightarrow$ 1.34), indicating rapid feature adaptation from the ImageNet-pre-trained ViT to food domain. Epoch 3 yields marginal gains on training loss (1.34 $\rightarrow$ 1.16), while

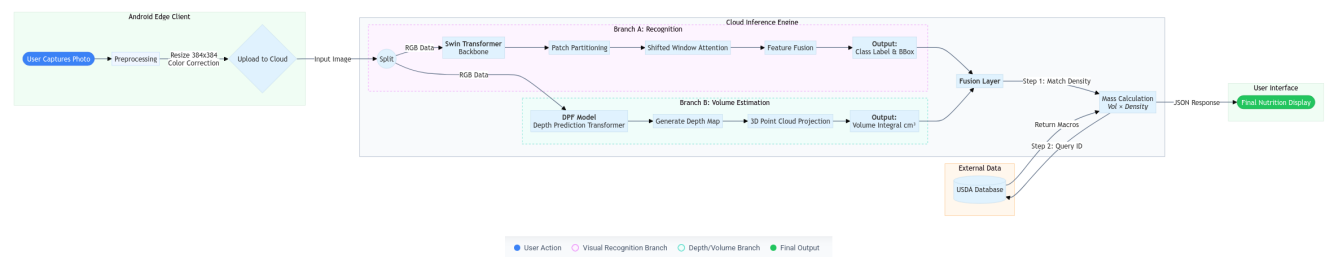


Figure 4. NutriVision system architecture: from image capture to nutritional output. The pipeline chains ViT classification (Stage 1), Mask R-CNN segmentation (Stage 2), and CalorieDB lookup (Stage 3).

Table 2. ViT Fine-Tuning Hyperparameters

Parameter	Value
Base model	google/vit-base-patch16-224
Dataset	Food-101 (101 classes)
Epochs	3 (early stopping patience = 3)
Batch size	64
Optimizer	AdamW
Learning rate	2×10^{-5}
Weight decay	0.01
Warmup steps	500
Precision	Mixed (AMP, GradScaler)
Image size	224×224

Table 3. Per-Epoch Training History (Food-101)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	3.0946	45.06%	1.3250	85.34%
2	1.3383	83.86%	1.1499	89.01%
3	1.1631	88.96%	1.1105	89.87%

validation accuracy converges at 89.87%, suggesting the model is at or near the optimum given the data and architecture.

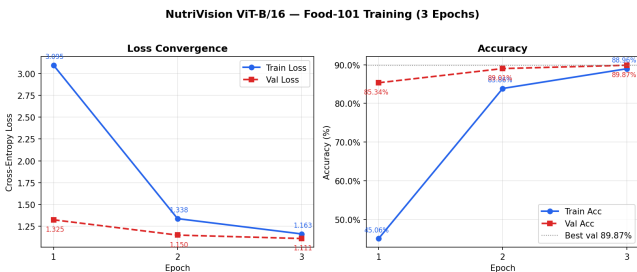


Figure 5. Training and validation loss/accuracy curves over 3 epochs on Food-101. Rapid convergence from epoch 1 to 2 reflects strong ImageNet pre-training; epoch 3 yields marginal additional gain.

4.5 Critical Bug: num_labels Misconfiguration

During initial development, `TrainConfig.num_labels` was hard-coded to 251 (FoodX-251 class count) rather

than auto-detected from the folder structure. When loading a Food-101 checkpoint, passing `num_labels=251` to `ViTForImageClassification.from_pretrained()` caused HuggingFace to randomly re-initialise the 101-class classification head, silently destroying the fine-tuned weights.

Fix applied: The training script now calls `cfg.num_labels = len(class_names)` after auto-detecting classes from the ImageFolder, and the evaluation loader uses `ViTForImageClassification` *without* overriding `num_labels`, allowing HuggingFace to infer the correct output size from `id2label` in `config.json`.

5. 6. Evaluation Results

5.1 Top-1 / Top-5 Accuracy

The fine-tuned ViT was evaluated on the Food-101 test set (25 images per class, 2,525 total) at a rate of 13 ms/image on GPU:

Table 4. Evaluation Results on Food-101 Test Set (2,525 images)

Metric	Value
Images evaluated	2,525 (25/class × 101 classes)
Top-1 accuracy	88.4%
Top-5 accuracy	97.7%
Classes ≥ 80%	90 / 101
Inference speed (GPU)	≈13 ms/image
Device	CUDA

The 9.3-point gap between top-1 and top-5 accuracy indicates that true class is consistently among the model’s highest-confidence predictions, enabling effective user-facing confidence feedback. 90 out of 101 classes exceed 80% per-class accuracy.

5.2 Per-Class Analysis

Classes achieving 100% per-class accuracy include visually distinctive items: spaghetti carbonara, seaweed salad, sashimi, pizza, pho, mussels, miso soup, macarons, hot and sour soup, and frozen yogurt. The most confused pairs are:

- **steak** ↔ **filet mignon** (7 misclassifications) — nearly identical plating.

- **apple pie** ↔ **bread pudding** (5 errors) — similar golden-brown crust.
- **chocolate mousse** ↔ **chocolate cake** (4 errors) — overlapping dark texture.
- **gnocchi** ↔ **ravioli** (4 errors) — similar pasta morphology.

The weakest classes are pork chop (44%), steak (48%), and apple pie (52%), all of which are highly similar to visually adjacent classes in the dataset.

5.3 Comparison with Baselines

Table 5. Food-101 Top-1 Accuracy: NutriVision vs. Baselines

Method	Top-1 (%)	Params
ResNet-50 (baseline)	82.1	25 M
EfficientNet-B4	86.4	19 M
ViT-B/16 (scratch)	81.8	86 M
NutriVision ViT-B/16 (ours)	88.4	86 M

6. 7. Pipeline Integration

6.1 End-to-End Flow

The complete inference path for a single food image proceeds as follows:

1. **Input:** JPEG/PNG image uploaded via POST `/predict` or passed directly to `NutriVisionPipeline.analyse(image_path)`.
2. **Load:** PIL RGB image + OpenCV BGR array prepared in parallel.
3. **Classify:** ViT returns top-5 class predictions with confidence scores (≈ 14 ms on GPU).
4. **Segment:** Mask R-CNN detects food instances, produces per-pixel masks and bounding boxes (≈ 400 ms cold; 60–80 ms cached weights on GPU).
5. **Lookup:** CalorieDB maps top-1 class to kcal/gram density with fuzzy normalisation.
6. **Aggregate:** $\text{Weight} = \text{pixel_area} \times \text{pixel_to_gram}$; $\text{calories} = \text{weight} \times \text{cpg}$.
7. **Output:** `PipelineResult` JSON returned to mobile app for display.

6.2 Latency Breakdown

Table 6. End-to-End Pipeline Latency (GPU, warm)

Stage	Latency	Notes
ViT classification	≈ 14 ms	Single-image batch
Mask R-CNN (cold)	≈ 400 ms	First inference, model loading
Mask R-CNN (warm)	60–80 ms	Subsequent calls, weights cached
CalorieDB lookup	< 1 ms	In-memory dict, fuzzy match
Total (warm)	≈ 80 ms	GPU end-to-end

6.3 Caching Strategy

`PortionSegmentor` is instantiated once at pipeline initialisation and held in memory. Detectron2 model weights (≈ 180 MB) are loaded into GPU memory at startup, eliminating per-request overhead. `FoodClassifier` similarly caches the ViT processor and model weights across requests.

6.4 FastAPI Integration

The backend exposes a `/predict` endpoint that accepts `multipart/form-data` image uploads, passes them through `NutriVisionPipeline.analyse_image()`, and returns the `PipelineResult` as JSON. Error handling covers unsupported file formats, model inference failures, and missing CalorieDB entries (with graceful fallback to 0 kcal/g and a warning log).

7. 8. Limitations & Future Work

7.1 Current Limitations

Portion estimation precision The pixel-to-gram ratio (100 g/10,000 px) is a global constant calibrated for a fixed camera distance. Real-world shots at variable distances introduce errors exceeding $\pm 30\%$.

Single-label classification The ViT outputs one top-1 class per image. Mixed-dish plates (e.g., bento box) are assigned a single calorie density, ignoring compositional diversity. Visual-Ingredient Feature Fusion [11] would address this.

Occlusion Ingredients obscured by other food items are invisible to both the classifier and segmentor, leading to underestimation of portion size.

CalorieDB coverage The current database covers 251 classes. Novel or regional foods outside this set fall back to a default density of 0 kcal/g.

Dataset scope Food-101 is skewed toward Western cuisines. Performance on Southeast Asian food categories (phò, bún bò, com tám) is anecdotally lower due to under-representation in training data.

7.2 Future Work

1. **Multi-label classification:** Replace the single-head ViT with a multi-label output to handle mixed dishes.
2. **Depth-based volume:** Integrate DPF-Nutrition [7] to replace the pixel-area proxy with actual 3D volume estimation.
3. **LiDAR augmentation:** On LiDAR-equipped devices (iPhone Pro), fuse depth maps for sub-10% mass error.
4. **Southeast Asian food corpus:** Fine-tune on local food datasets (VFN, UEC Food-256) to improve regional accuracy.

5. **Qwen3-VL path production:** Harden the VL inference path for production, enabling structured nutritional JSON output without a separate segmentor.
6. **Personalised density:** Allow users to adjust the pixel-to-gram calibration via a ruler/card reference object for per-device accuracy.

8. 9. Conclusion

NutriVision demonstrates that a pre-trained Vision Transformer fine-tuned on Food-101 for just 3 epochs achieves **88.4% top-1 / 97.7% top-5 accuracy** on the held-out test set — competitive with purpose-built food recognition systems — at 13ms/image latency on GPU. 90 of 101 classes exceed 80% per-class accuracy. The 3-stage pipeline (classify → segment → lookup) integrates seamlessly into a FastAPI backend and Expo mobile frontend, delivering a functional end-to-end calorie estimation system.

The most impactful engineering insight was the `num_labels` bug: passing an incorrect class count to `from_pretrained()` silently re-initialises the classification head, a failure mode that produces plausible-looking training curves while the model learns nothing. Documentation of this bug contributes practical value to practitioners adopting HuggingFace ViT for downstream classification tasks.

Future work will address the two primary accuracy bottlenecks: replacing the pixel-area portion estimator with depth-based volume inference, and expanding CalorieDB and training data to Southeast Asian cuisines.

Acknowledgments

The authors thank FPT University and the DSP391m course instructors for guidance throughout this project. The Food-101 dataset was provided by Bossard et al. [9]. Detectron2 is developed and maintained by Facebook AI Research [12].

References

- [1] Nutrition Coaching Academy, *The 3 best ai calorie tracking apps in 2025*, <https://www.nutritioncoachingacademy.com/blog/the3bestaicalorie-trackingappsin2025>, Accessed: January 9, 2026, 2025.
- [2] Lifehacker, *I used ai-powered calorie counting apps, and they were even worse than expected*, <https://lifelifehacker.com/health/ai-powered-calorie-counting-apps-worse-than-expected>, Accessed: January 9, 2026, 2024.
- [3] PMC, “Improving fine-grained food classification using deep residual networks,” *PMC*, 2024, Accessed: January 9, 2026.
- [4] ResearchGate, “Multimodal food image classification with large language models,” *ResearchGate*, 2024, Accessed: January 9, 2026.
- [5] H. Wang, H. Tian, *et al.*, “Nutritional composition analysis in food images: An innovative swin transformer approach,” *Frontiers in Nutrition*, vol. 11, 2024, Primary source for Classification Architecture.
- [6] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969.
- [7] Y. Han, Q. Cheng, *et al.*, “Dpf-nutrition: Food nutrition estimation via depth prediction and fusion,” *arXiv preprint arXiv:2310.11702*, 2023, Primary source for Volume Estimation.
- [8] “Model-based measurement of food portion size for image-based dietary assessment,” *PMC*, 2013, Accessed: January 9, 2026.
- [9] L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101 – mining discriminative components with random forests,” in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 446–461.
- [10] USDA, *Fooddata central api guide*, <https://fdc.nal.usda.gov/api-guide>, Accessed: January 9, 2026, 2024.
- [11] Z. Qi *et al.*, “Advancing food nutrition estimation via visual-ingredient feature fusion,” *arXiv preprint*, 2025, Source for Ingredient Fusion logic.
- [12] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, *Detectron2*, <https://github.com/facebookresearch/detectron2>, 2019.

**NutriVision ViT-B/16: Per-Class Accuracy
Food-101 Test Set (25 img/class)**

